Canvas

Basic Usage

MDN

Canvas: what?

- An HTML tag → <canvas></canvas>
- A built-in JavaScript API for drawing graphics and animations
- Read the MDN docs
- MDN Tutorial

Basic usage of canvas

Create a <canvas> element in HTML

```
<canvas class="drawing-surface" width="150" height="150"></canvas>
```

Address the element in JavaScript

```
const $canvas = document.querySelector(`.drawing-surface`);
const ctx = $canvas.getContext(`2d`);
```

Start drawing using the CanvasRenderingContext2D context

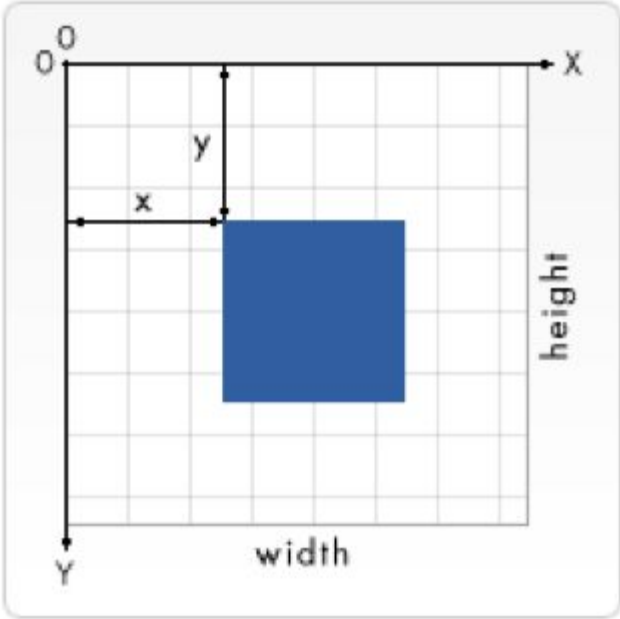
Demo 01 - basic usage

Drawing shapes

MDN

The Grid

- Point (0,0) is in the upper left corner
- All elements are positioned to this point
- Move this point using [transformations]



Drawing rectangles

- **fillRect(x, y, width, height)** → draws a filled rectangle in black MDN
- **fillStyle** → set a color for the object MDN
- **strokeStyle** → sets a color for the stroke MDN
- **lineWidth** → sets the width of the stroke MDN
- **strokeRect(x, y, width, height)** → draws a rectangular outline MDN
- **clearRect(x, y, width, height)** → clears the specified area making it transparent MDN

Colors

- fillStyle = color
  - sets the style used when filling shapes.
- strokeStyle = color
  - set the style for shapes' outlines.

```
ctx.fillStyle = `orange`;
ctx.fillStyle = `#FFA500`;
ctx.fillStyle = `rgb(255,165,0)`;
ctx.fillStyle = `rgba(255,165,0,1)`;
```

Drawing paths

MDN

Drawing paths

- **beginPath()** → creates a new path. Once created, future drawing commands are directed into the path and used to build the path up.
- **closePath()** → Closes the path so that future drawing commands are once again directed to the context.
- **moveTo(x,y)** → Moves the starting point of a new sub-path to the (x, y) coordinates.
- **lineTo(x,y)** → Connects the last point in the subpath to the x, y coordinates with a straight line.
- **stroke()** → Draws the shape by stroking its outline
- **fill()** → Draws the shape by filling the shape

Demo 03 - drawing paths

closePath()

necessary when:

- You want to create a **closed shape** that needs to be filled

not necessary when:

- You’re drawing open paths like lines or curves
- You’re creating custom shapes where you don’t want the last point connected to the first

Drawing circles

MDN

Drawing circles

- **arc(x, y, radius, startAngle, endAngle)** → adds an arc to the path that is centered at (x, y) position with radius r starting at startAngle and ending at endAngle (clockwise).

Demo 04 - drawing circles

Drawing text

MDN

Drawing text

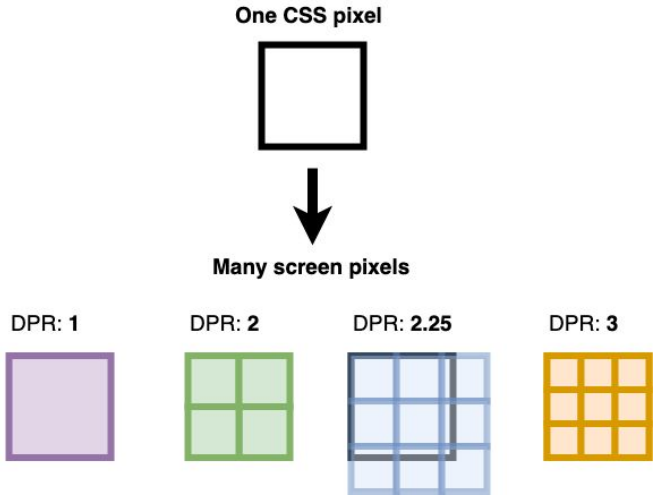
- fillText(text, x, y) → fills a given text at the given (x,y) position
- strokeText(text, x, y [, maxWidth]) → strokes a given text at the given (x,y) position.

Demo 05 - drawing text

Window.devicePixelRatio

Window.devicePixelRatio

- Canvas is a bitmap that looks blurry because:
  - CSS pixel is not the same as a device pixel
  - On Retina screens the Device pixel ratio is 2, meaning 1 CSS pixel actually represents 4 Device pixels.
- So we need to double every pixel to get sharp results
- Make the canvas twice as large (width & height attribute)
- And scale it down to the original size with inline CSS.



Demo 06 - device ratio

Animations

Animations

- For example: moving an object
- we need some kind of repetition to adjust the position

```
const animate = () => {
    xPos++;
    size += 0.1;

    // repeat the animation
};
```

Functions with repetition

- **setInterval(function, delay)** → Starts repeatedly executing the function specified by function every delay milliseconds.

- **setTimeout(function, delay)** → Executes the function specified by function in delay milliseconds.

- **requestAnimationFrame(callback)** → Tells the browser that you wish to perform an animation and requests that the browser call a specified function to update an animation before the next repaint.

Functions with repetition

- setInterval(function, delay) → Starts repeatedly executing the function specified by function every delay milliseconds.

- setTimeout(function, delay) → Executes the function specified by function in delay milliseconds.

- **requestAnimationFrame(callback)** → Tells the browser that you wish to perform an animation and requests that the browser call a specified function to update an animation before the next repaint.

requestAnimationFrame

- Is synchronized with the repaint function of the browser
- Is synchronized with the refresh rate of your screen
- Accelerates / slows down / pauzes automatically
- Saves battery time

CONCLUSION: always use requestAnimationFrame

Using images

MDN

Drawing images

```
// set image source
image = new Image();
image.src = './assets/protest.jpeg';

// draw image on canvas when loaded
image.addEventListener('load', () => {
  drawImage(image);
});
```